

DRIFT

A trading bot whose risk rules you can verify

Off-chain quant research · on-chain risk gate on BNB Chain

Live demo

drift-macroguard.vercel.app/macroguard

Dex's contract on BSC Testnet

[0x8b09ebB85Be8Ed55Bb5132d29eABc567c42aa83D](https://bscscan.com/address/0x8b09ebB85Be8Ed55Bb5132d29eABc567c42aa83D)

Indonesia Web3 Hackathon 2026 · Dex Bennett

The problem

Trading bots ask users to trust risk rules nobody can see.

Settings change silently; off-chain decision logs can be rewritten.

The promise

Anyone can check what DRIFT's bot is allowed to do right now,
and every recorded decision is public on BscScan.

Who needs it, and how it could pay

HYPOTHESIS: not yet validated with users; no market figures claimed

Who needs it

Retail traders who follow or rent bots: proof that the risk stop is real

Bot builders and signal sellers: a record of discipline a screenshot cannot fake

Communities and auditors: check a bot's rules without trusting its operator

How it could pay

A risk gate per bot: its own MacroGuard plus a public panel, as a monthly service

The panel link as a trust badge for strategy sellers

No custody and no performance fee; on-chain cost per decision stays tiny (slide 7)

How DRIFT works

1 Signal

Python engine: four strategies, point-in-time, no look-ahead

2 Ask the gate

`MacroGuard.allowed(signal)` on BSC Testnet: regime + 20% halt

3 Trade

Runner places the order on Bybit testnet (vetoed signals go Flat)

4 Record

`recordDecision(...)` leaves a public receipt; a breach halts it

Periwinkle: off-chain engine · white: on-chain contract

The contract never executes orders; it is the public rule and the public record.

The panel, and asking the contract

MACROGUARD · BNB SMART CHAIN TESTNET · CHAIN 97

What is the bot allowed to do right now?

DRIFT's quant engine runs off-chain. Its risk gate lives in a public contract. This page shows the contract's state as read from BNB Chain. It never signs or sends a transaction.

read at 13:51:24 via data-seed-prebsc-1-s1.bnbchain.org:8545 [Refresh](#)

LIVE RISK STATE

Running — no drawdown halt

Open. The contract allows Long, Short and Flat right now.

Market regime stored on-chain: Neutral

Risk off
Long vetoed

Neutral
All signals open

Risk on
All signals open

Who changes the regime?
Only the agent key, via setRegime, from the engine's macro classifier.

What triggers a halt?
A recorded decision with drawdown past the threshold. The contract flips it itself.

ON-CHAIN PROOF

LIVE READ · PUBLIC RPC · BLOCK 134,393,170

Contract
0x8b09ebB8_c42aa83D [copy](#)

Agent (only key that can write)
0x2B07Af85_C6714b81

Halt threshold
20% (2000 bps)

Decisions logged
2

Open on BscScan ↗

Deploy tx ↗

WHY A SIGNAL IS ALLOWED OR BLOCKED

contract call allowed(signal)

LONG
Allowed
Open while the regime is not Risk off.

SHORT
Allowed
Only a drawdown halt would block it.

FLAT
Allowed
Always allowed — It reduces exposure.

DECISION TRAIL · VERIFIED RECEIPTS

smoke test 2026-09-30 - every row opens BscScan

01 Deploy
constructor(maxDrawdownBps = 2000)

Neutral · not halted · 0 decisions

status 1 #133,995,398 · 0x2d8cce_c044 ↗

Live regime, halt, 20% line and decisions,
read from BSC Testnet at a named block

ASK THE CONTRACT · SIMULATION

eth_call · nothing signed, nothing written

What would MacroGuard decide if the bot reported this?

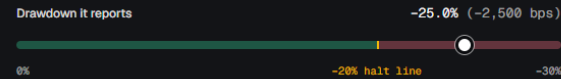
Pick a signal and the drawdown the bot would report. Your browser sends one eth_call of recordDecision to the live contract, as if from the agent address. It is a simulation: nothing is signed or written, no transaction is created and no gas is spent. The answer comes from the contract on BSC Testnet, not from this page.

Signal the bot wants

Long
buy

Short
sell short

Flat
exit



Ask the live contract

Blocked Long at -25.0%

At -25.0% the drawdown is at or past the 20% halt line, so the contract halts itself first. After a halt only Flat passes.

```
call recordDecision("BTCUSD*", Long, 0, -2500)
from agent 0x2B07_4b81 · unsigned
to MacroGuard 0x8b09_a83D ↗
read at block 134,605,519 via data-seed-prebsc-1-s1.bnbchain.org:8545
```

Simulation only: the node ran the call and discarded the result. The on-chain decision count is unchanged.

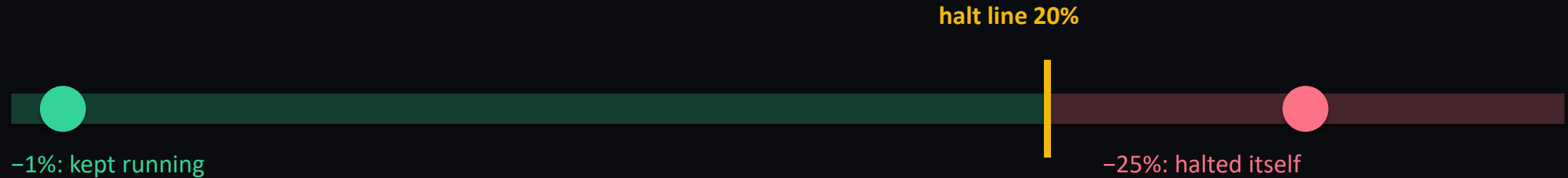
▼ Replay it with Foundry's cast

```
cast call 0x8b09ebB859e0Ed558b5132d29eABc567c42aa83D
'recordDecision(string,uint8,uint256,int256)(bool)*' --from
0x2B07Af854068042664074781Af36163aC8714b81 --rpc-url https://bsc-
testnet-rpc.publicnode.com -- BTCUSD* 1 0 -2500
```

[copy](#)

Long at -25%: Blocked, answered by the live contract
via eth_call from the agent address (simulation)

-25% recorded: the contract halted itself



#134,042,196	setRegime(RiskOff)	Long blocked; Short and Flat open
#134,042,256	recordDecision(BNB, Short, -1%)	Decision #1, inside the line
#134,042,283	recordDecision(BNB, Short, -25%)	Halted event: only Flat allowed
#134,042,318	resume()	Agent-only, on the record
#134,042,328	setRegime(Neutral)	Long allowed again; 2 decisions on-chain

Every row is a transaction with receipt status 1 on BscScan · block 134,042,283 is the halt

Why BNB Chain

≈ 0.00000034 tBNB

per recorded decision: 33,189–34,323 gas
at 0.1 gwei (our receipts, BSC Testnet)

≈ 0.00008 tBNB a day

for an hourly bot (24 decisions);
deploy: 449,207 gas ≈ 0.000045 tBNB

What else it gives us

EVM: Foundry tests and Sourcify verification on chain 97

Public RPCs a browser can call: the panel needs no backend

Five smoke-test transactions within 132 blocks

Mainnet (chain 56) is a config switch; not deployed there

Testnet gas price, not a mainnet quote. Mainnet is on the roadmap, after an audit.

Evidence and current limits

30 / 30

contract tests: unit, fuzz, invariant
100% coverage of MacroGuard.sol

48

offline engine tests;
no look-ahead tested as a property

15

web tests: contract reads and the
what-if encoder against cast

exact match

source verified on Sourcify
6 receipts, all status 1

Limits, said plainly

The contract never executes orders · the runner fails open if the RPC is down · the agent can `resume()` at once
The drawdown is reported by the agent · no live bot tick and no profit claimed; backtests are research

What Dex added

Upstream DRIFT project

Quant engine, cockpit and MacroGuard.sol

7 contract tests

Built for a Mantle hackathon track (June 2026)

Migrated to BNB Chain in bcc-ukdw/seed-bnb
(commit 52671ce, 29 Sep 2026)

Dex's work in this fork

Own BSC Testnet deploy + 5-tx smoke test (status 1)

Source verified on Sourcify (exact match)

Public panel: live browser reads, receipts, limits

Ask the contract: `eth_call` what-if, nothing signed
+23 contract tests, 48 engine tests, 15 web tests, CI

Labelled Binance data fallback · threat model

Landing, visual system, PRD, this deck

Code: github.com/Stylenecy/seed-bnb/tree/dex/drift/drift

Roadmap

Not built yet; listed in order of trust gained per step

Resume through a multisig or timelock

Fail-closed mode for the runner

Signed regime verdicts (EIP-712) with an input hash

Attested equity, so the drawdown is not self-reported

Record the raw signal plus a veto flag

Agent identity (ERC-8004), then mainnet after an audit

Check it yourself

Live demo <https://drift-macroguard.vercel.app/macroguard>

Code <https://github.com/Stylenecy/seed-bnb/tree/dex/drift/drift>

Contract <https://testnet.bscscan.com/address/0x8b09ebB85Be8Ed55Bb5132d29eABc567c42aa83D>

Source <https://repo.sourcify.dev/97/0x8b09ebB85Be8Ed55Bb5132d29eABc567c42aa83D>